# K-Means-Clustering Accuracy vs. Random Selection Accuracy

**Danny Quang**

## Abstract

This project explores my idea for coordinate descent in in a standard unconstrained optimization problem. Specifically, I explore and implement my algorithm on a binary logistic regression problem with the wine dataset.

We try random selection, where we select random coordinates or points as our descent directions, and use a logistic loss function. Random selection is a baseline method, and our method to choose the strongest gradient aims to increase efficiency by choosing better coordinates, or directions, for our logistic regression method.

We can see that our maximum gradient strategy converges quicker whereas random selection is not as smooth and not as directly. We also take a look at the conditions for convergence and potential improvements.

## 1 Description

In my experiment, I implement and run the maximum gradient coordinate selection algorithm to improve the gradient direction selection. We want to consider the standard optimization of minimizing the loss L:

$$\min L(w) \tag{1}$$

We use logistic regression in this goal of minimizing L with logistic loss:

$$L(w) = \frac{1}{n}\Sigma \ln(1 + \exp(-y_i {x_i}^T w)) \tag{2}$$

where each label $y_i \in \{-1, 1\}$.

### 1.1 (a) Coordinate Selection

#### 1.1.1 Random Selection

First, I run a baseline algorithm where for each iteration I pick random directions to serve as the optimal gradient direction so that our update heads in the direction of the random direction. This works by choosing a random feature of the entire feature set and updating the gradient of that coordinate at the update step.

#### 1.1.2 (b) Largest Gradient Coordinate Selection

Then, I implement the algorithm where for each iteration, we compute the full logistic gradient $L(w)$ and choose the coordinate c with the largest partial derivative. We do this by finding the

$$argmax|L(w)| \tag{3}$$

### 1.2 (b) Largest Gradient Selection

Once we choose a coordinate, whether it is random or the largest gradient coordinate/feature, we perform an update to the weight for that coordinate/feature c:

$$w[c] = w[c] - learningrate - gradient[c] \tag{4}$$

or more specifically the change as:

$$\triangle = -\eta *_c L(w) \tag{5}$$

### 1.3 Differentiability

These methods uses gradients that do require $L(\cdot)$ to be differentiable. So, the second order derivatives, or hessians, do need to be continuous. There do exist subgradient functions that keep track of the best points found so far that do not need $L(\cdot)$ to be differentiable.

## 2 Convergence

I think that as long as the step size isn't too large to the point that it steps over entire minimum valleys or bounces back and forth the sides of a valley, then our coordinate descent functions will eventually converge to a global minimum on a convex and differentiable $L(\cdot)$ function. If we choose the

largest gradient or random weight update then we will converge to a minimal $L$ value.

For logistic regression, we know it is strictly convex if no two points are identical, so it will converge to a global minimum.

## 3 Experimental Results

Our Wine dataset consists of 130 samples where 59 belonged to the first class and 71 to the second class. We standardize the features.

### 3.1 (a) Standard Logistic Regression, Optimal Loss L*

After running scikit-learn's LogisticRegression without regularization, we find an optimal loss $L*$ of 0.04618584401952307.

### 3.2 (b) Largest Gradient Coordinate Descent

We implement that largest gradient coordinate selection as:

Algorithm: Largest Gradient Coordinate Selection

INPUT: Training data X, training labels y, maximum iterations max_iter, and learning rate
OUTPUT: Weights w, losses

0. Initialize w to all zeros for each dimension d.

1. For each iteration up to max_iter, get the current logistic gradient with X and y with weight w, or gradients = logisticGradient(w, X, y)
- Get c, the argmax of the abs. value of the gradients, or the largest/strongest gradient
- Update w[c] with -learning rate * gradients[c]
- Track the logistic loss of the current w, X, y. Or, losses.append(logistic_loss(w, X, y))

2. Return corresponding weights and losses for the largest gradients at each step.

### 3.3 (c) Random Coordinate Descent

Algorithm: Random Selection

INPUT: Training data X, training labels y, maximum iterations max_iter, and learning rate
OUTPUT: Weights w, losses

0. Initialize w to all zeros for each dimension d.

1. For each iteration up to max_iter, get the current logistic gradient with X and y with weight w, or gradients = logisticGradient(w, X, y)
- Get j, a random coordinate that will be out weight step update and gradient direction.
- Update w[c] with -learning rate * gradients[c]
- Track the logistic loss of the current w, X, y. Or, losses.append(logistic_loss(w, X, y))

2. Return corresponding weights and losses for the largest gradients at each step.

As we can see, the only difference between random and largest gradient selection is the weight update. Our figure demonstrates that our largest gradient selection in coordinate descent immediately gets a lower loss than random and continues to maintain a lower loss than random selection, which is consistent with our expectations.

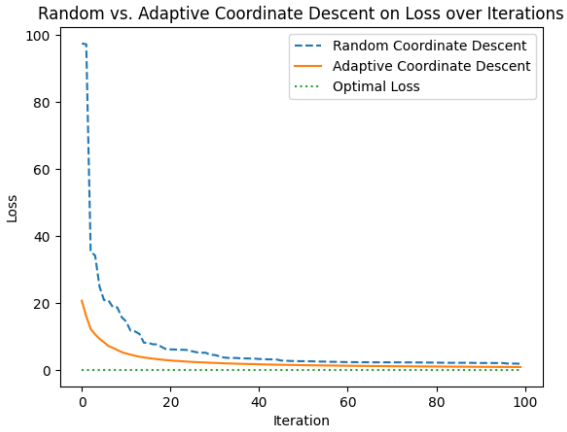


Figure 1: Largest Gradient Coordinate Selection Performance vs. Random Selection Performance Relative to Optimal Loss L*

## 4 Critical Evaluation

As the results show, largest gradient coordinate selection shows an improvement over random selection. It converges more rapidly while the random selection doesn't converge in as smooth of a curve though it still does eventually converge. Both of the methods eventually converge to $L(*)$ which verifies the logistic regression correctness.

## 5 Conclusion

We have shown that choosing a random or largest gradient coordinate can approach the optimal $L*$ in logistic regression coordinate descent. By picking the optimal gradient as our update step, we can

approach the optimal $L*$ quicker and get closer with less iterations.

## 6 Sparse Coordinate Descent

We can do a k-sparse coordinate descent method by just adding an extra step after our update step to prune the weakest coordinates by magnitude, should the update exceed k nonzero values. This effectively keeps the k largest magnitude coefficients and sets the remaining coefficients to zero until convergence. This may not always find teh best k-sparse solution even if $L(\cdot)$ is convex. This is because coordinate descent makes greedy local updates and may not find the best global k-sparse subset. The features that are kept early on may not actually be part of the globally optimum k-sparse set. However, it may find a good approximation and thus it can be applied to a non-convex problem.